Ders 16 Genişletilmiş Notu: İş Parçacıkları: Çoklu İş Parçacığı Modelleri, Pthreads, Amdahl Yasası ve İş Parçacığı Sorunları

Ders 16 Genişletilmiş Notu: İş Parçacıkları: Çoklu İş Parçacığı Modelleri, Pthreads, Amdahl Yasası ve İş Parçacığı Sorunları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı
- 2. Motivasyon: Neden İş Parçacığı?
- 3. Örnekler: Metin Düzenleyici ve Web Sunucusu
- 4. Çok İş Parçacıklı Yapının Faydaları
- 5. Çok Çekirdekli Programlama
- 6. Tek ve Çok İş Parçacıklı Süreçlerin Yapısı
- 7. Amdahl Yasası
- 8. Kullanıcı Seviyesi ve Çekirdek Seviyesi İş Parçacıkları
- 9. Çoklu İş Parçacığı Modelleri
- 10. İş Parçacığı Kütüphaneleri
- 11. Pthreads Örneği: 1'den N'ye Toplam
- 12. Windows ve Java İş Parçacıkları
- 13. Örtük İş Parçacığı Kullanımı
- 14. İş Parçacığı Kullanımındaki Sorunlar
- 15. Örnek Sorular

1. Dersin Planı

Ders Zoom üzerinden yapıldı. Konu, ders kitabının (Silberschatz, *Operating System Concepts*) iş parçacıkları (threads) bölümüydü. Türkçede bunlara işlem parçacığı da denir. Hoca derste şu soruların konuşulacağını söyledi:

- İş parçacığı nedir, ne işe yarar?
- İşletim sistemi tarafında nasıl gerçeklenir (implement edilir)? Gerçekleme sırasında hangi unsurlar düşünülmelidir?
- Örnek sistemlerde ve uygulama geliştirme tarafında bu yapılar nasıl işler?
- İş parçacığı kullanımının getirdiği karmaşıklıklar için hangi çözümler üretilmiştir?

Son madde için hocanın verdiği ilk örnek sistem çağrılarıydı. Bir iş parçacığının içinden çoğu sistem çağrısını kullanmak sorun çıkarmaz. Fakat `fork` ve `exec` gibi bazı özel çağrılarda ne olacağı açık değildir. Bu alternatifler dersin sonunda konuşuldu.

2. Motivasyon: Neden İş Parçacığı?

Hoca motivasyonu, dersin genelindeki ana hedefe bağladı: Bilgisayar sisteminin pahalı ve önemli bir kaynağı olan işlemci zamanını etkin kullanmak. Bunun için çoklu işleme (multi-

processing) ve zaman paylaşımı (time-sharing) yapıyoruz. Çünkü tek bir kullanıcının tek bir süreci, hatta bir grup süreci, işlemciyi her zaman meşgul edemez. Birden fazla süreç olunca işlemci zamanı daha çok kullanılır.

Bir süreç (process), belli bir adresten başlayıp kod bloğunu ardı ardına çalıştıran tek bir akış sağlar. Bu akış her zaman yeterli değildir. O zaman ne yaparız? Benzer işi yapan bir süreç daha oluştururuz. Fakat yeni bir süreç oluşturmak, süreci oluşturan bütün bileşenleri yeniden oluşturmak demektir. Hoca burada ikinci bölümdeki süreç yapısı şeklini hatırlattı: Sıfırdan başlayan adres uzayında yığın (stack), yığın bellek (heap), veri bölümü (data section) ve program kodunun bulunduğu metin bölümü (text section) vardır. `fork` ile yaptığımız şey de bu bölümlerin hepsini aynen kopyalamaktı.

Oysa asıl istediğimiz, var olan akışa benzer yeni bir akış oluşturmaktır. Bir akışı belirleyen temel unsurlar şunlardır:

- Hangi komutta olduğumuzu söyleyen program sayacı (program counter).
- Bir fonksiyon çağrısında parametrelerin ve dönüş adresinin basıldığı yığın.
- Yazmaçlar (registers).

Bir akış oluşturmak için bunların dışında ekstra bir şeye ihtiyaç yoktur. O halde çok sayıda süreç oluşturmak yerine, bir sürecin içinde yalnız bu küçük yapıları çoğaltarak birden fazla akış oluşturabiliriz. Ortaya çıkan yapı daha hafiftir. Hocanın ifadesiyle: Bir süreç için bir birim bellek harcıyorsak, iki süreç için iki birim harcarız. Ama iki iş parçacığına sahip tek bir süreç için harcanan iki birim olmaz, çok daha az olur. Amaç, daha az kaynakla daha çok iş yapabilmektir.

3. Örnekler: Metin Düzenleyici ve Web Sunucusu

3.1 Metin Düzenleyici

Hoca daha önce çoklu süreç için tarayıcı örneğini kullandıklarını hatırlattı. Bu bölümdeki örnek bir metin düzenleyiciydi. Bir metin düzenleyicide hızlı hızlı kullanıcı girdisi alınır, ekran güncellenir, arada yazım hatası denetimi yapılır. Bunları tek süreç ve tek akışla yapmak gerekir. Çok iş parçacıklı yapıldığında ise her biri için bağımsız olarak sorumlu ayrı bir akış olur. Böylece hem daha etkin hem de daha etkileşimli bir yapı kurulur. Hoca, bugün kullandığımız modern işletim sistemlerinin hepsinin çoklu iş parçacığı desteği sağladığını belirtti.

3.2 Web Sunucusu

Hocanın “hep verdiğim örnek” dediği örnek sunucu sistemiydi. Tarayıcıyla bir web sitesine bağlandığımızda site bir cevap üretir ve tarayıcıda görünür. Ama siteye bağlanan tek kişi biz değilizdir, farklı adreslerden pek çok kişi bağlanır.

- **Tek süreç:** Herkes kuyruğa girip beklemek zorundadır. Bu etkin değildir.
- **Çoklu süreç, önceden oluşturulmuş süreçler:** Her istek önceden oluşturulmuş bir sürece yönlendirilir. Süreç hazır beklediği için gecikme düşüktür. Ama yapı statiktir, süreç sayısını dinamik olarak artırıp azaltmak mümkün değildir.
- **Çoklu süreç, istek geldikçe süreç oluşturma:** Ortam daha dinamiktir. Fakat iki sorunu vardır. Biri kötüye kullanıma açık olmasıdır: Birisi binlerce istek göndererek bir anda binlerce süreç oluşturmanıza neden olabilir. Diğeri, süreç istek geldiği anda oluşturulduğu için ekstra gecikme olmasıdır.
- **İş parçacığı:** Her istek için süreç yerine bir iş parçacığı oluşturulur ve gelen istekle o ilgilenir. İş parçacığı daha küçük bir yapı olduğu için oluşturulması daha hızlıdır.

4. Çok İş Parçacıklı Yapının Faydaları

Hoca, şimdiye kadar konuşulanları dört fayda altında topladı:

- **Duyarlılık (responsiveness):** Daha duyarlı bir yapı kurulur. Etkileşimin önemli olduğu yerlerde bu önemlidir.
- **Ekonomi (economy):** Bir süreci kopyalamak çok daha fazla kaynak ve bellek gerektirir. İş parçacığı bu anlamda daha ekonomiktir.

- **Ölçeklenebilirlik (scalability):** Daha ekonomik olduğu için ölçeklenebilirlik açısından da daha avantajlıdır.
- **Kaynak paylaşımı (resource sharing):** Süreçler arası iletişimde (inter-process communication) `fork` ile oluşan kopya süreçler bile birbirinden bağımsızdır. Haberleşmek için ileti aktarımı (message passing) ya da paylaşılan bellek (shared memory) gibi mekanizmalar gerekir. İş parçacıklarında ise yığınlar ve program sayaçları bağımsız olsa da veri bölümü, metin bölümü ve yığın bellek ortaktır. Bu yüzden iş parçacıkları aynı kaynaklara doğrudan erişebilir, ayrı bir haberleşme mekanizması gerekmez.

  Hocanın vurgusu: Ortak kaynağa doğrudan erişim bir artıdır, ama dikkat gerektirir. Paylaşılan bellekte olduğu gibi burada da işletim sistemi yalnız paylaşımın mekanizmasını sağlar. Ortak alanın düzgün kullanılması uygulama geliştiricinin sorumluluğundadır.

5. Çok Çekirdekli Programlama

5.1 Çok Çekirdekli Sistemler ve Paralelleştirme Yolları

Tek işlemci varken iş görece kolaydır: Tek bir kaynak vardır, süreçler ya da iş parçacıkları onun üzerinde değiştirilir. Bugün ise çoğunlukla tek bir işlemcimiz vardır ama içinde birden fazla çekirdek (core) bulunur. Nadiren de birden fazla işlemci vardır ve her birinde birden fazla çekirdek bulunur (multiprocessor system). Bu kaynağı kullanabilmek için programlamayı da buna uygun hale getirmek gerekir.

Hoca bir işi birden fazla süreç ya da iş parçacığına bölmenin iki temel yolunu anlattı:

- **Veriyi bölmek (veri paralelliği, data parallelism):** İş büyük bir veri üzerindeyse ve verinin farklı parçaları bağımsız olarak işlenebiliyorsa veri bölünür ve her parça ayrı işlenir. Hocanın deyişiyle bu, paralellik için "10 numara 5 yıldız" bir sebeptir.
- **Aktiviteleri bölmek (görev paralelliği, task parallelism):** Tarayıcı ve metin düzenleyici örneklerinde hep bunu yaptık: İşi birbirinden bağımsız aktivitelere ayırdık. Aktiviteler ayrı süreç ya da iş parçacıklarına bölününce, birden fazla çekirdek olduğunda aynı anda çalışabilir, yani paralellik elde edilir.

5.2 Zorluklar

Veri bağımlılığı. Bir verinin işlenmesi kendinden önceki, sonraki ya da aynı çerçevedeki verilere bağlıysa o veri serbestçe bölünemez. Bunu iyi analiz etmek gerekir. Hocanın örneği görüntü işlemeydi: Elinizde Full HD bir çerçeve var ve üzerinde 3×3, 5×5 ya da 7×7 bir çekirdek (kernel) gezdireceksiniz. Veriyi bölüp bir kısmını bir işlemciye, bir kısmını başka bir işlemciye verebilirsiniz. Ama bölmeyi çekirdek boyutuna göre yapmak zorundasınız. Çünkü bir noktayı hesaplarken üstteki ve alttaki satırları da kullanırsınız.

Test ve hata ayıklama. Çok süreçli ya da çok iş parçacıklı sistemlerde hatayı bulmak, ayıklamak ve test etmek kolay değildir. Tek akışta bir sonraki komutun ne olduğunu bilirsiniz. Birden fazla akışta ise her akış herhangi bir anda kendi içinde çok farklı bir yerde olabilir. Bu durum, birbiriyle çelişen ya da birbirinin çalışmasını bozan sonuçlar üretebilir. Örneğin on iş parçacığı varsa herhangi bir anda ne olabileceğine dair ihtimaller çok fazladır, hangisinin programın neresinde olduğunu bilemezsiniz.

5.3 Paralellik ve Eşzamanlılık

Hoca iki temel kavramı slayttaki şekil üzerinden ayırdı:

- **Paralellik (parallelism):** Farklı görevlerin gerçekten aynı anda çalışmasıdır. Örneğin birinci çekirdekte T1 çalışırken ikinci çekirdekte T2'nin çalışması.
- **Eşzamanlılık (concurrency):** İşlemci zamanı görevler arasında bölünür. Böylece görevler paralel çalışıyormuş, aynı anda ilerliyormuş izlenimi verilir.

5.4 Donanım İş Parçacıkları

İş parçacığı yapısı yaygınlaştıkça donanım seviyesinde de iş parçacığı desteği görülmeye başlandı. Hocanın örneği: Bir Intel işlemcide 8 çekirdek ya da 8 iş parçacığı desteği görünür. Ama 8 iş parçacığı desteği varsa genelde aslında 4 çekirdek vardır. Hoca bunu, çekirdeklerin işlemci zamanının bölünerek sanki 8 işlemci varmış gibi gösterilmesi olarak açıkladı ve orada da bir tür zamanlama çalıştığını söyledi.

> **Ders sonrası not:** Bu düzen, Intel'in Hyper-Threading adını verdiği eşzamanlı çoklu iş parçacığı (simultaneous multithreading, SMT) tekniğidir. Büyük olasılıkla burada yalnızca zaman paylaşımı yoktur: Her çekirdek iki donanım iş parçacığının komutlarını aynı anda yürütür ve çekirdeğin işlem birimlerini aralarında paylaştırır. Bu yüzden 4 çekirdek/8 iş parçacıklı bir işlemci, 8 çekirdekli bir işlemcinin başarımına ulaşmaz.

6. Tek ve Çok İş Parçacıklı Süreçlerin Yapısı

Hoca, şimdiye kadar söylenenleri tek ve çok iş parçacıklı süreç şeklinin üzerinden tekrar etti. Bir süreçte kod, veri, yığın bellek, açık dosyalar ve benzeri kaynaklar vardır. Bunun yanında bir akış vardır. Akışın ilerlemesi için gereken temel şeyler yığın ve yazmaçlardır.

Çok iş parçacıklı yapıda her iş parçacığının bağımsız ilerleyebilmesi için şunlar her iş parçacığında ayrı tutulur:

- yazmaç kümesi (register set)
- yığın
- program sayacı

Geri kalan her şey (kod, veri, yığın bellek, açık dosyalar) iş parçacıkları için ortaktır. İş parçacıkları arasında haberleşme yapmak isteniyorsa bu ortak alan üzerinden yapılabilir. Bu da kaynak kullanımında etkinlik getirir.

7. Amdahl Yasası

7.1 Neden İki Kat Hızlanmıyoruz?

Hoca şu durumu sordu: Sisteminizde bir işlemci var ve belli bir hızda çalışıyor. Sisteme bir işlemci daha eklediğinizde, programınızı buna uygun yazsanız bile başarımınız her zaman iki katına çıkmaz. Neden? Bu, Amdahl Yasası (Amdahl's Law) ile açıklanır ve programınızın ne kadarının seri, ne kadarının paralel olduğuyla ilgilidir.

Programı iki kısma ayıralım: seri kısım S ve paralel kısım. İkisinin toplamı programın tamamıdır, yani 1'dir. Dolayısıyla paralel kısım $1 - S$ olarak yazılır. Paralelleştirilebilen kısmı N çekirdeğe bölerseniz o kısım N kat hızlanır. Seri kısım ise bölünemez ve olduğu gibi kalır. 1'i bu toplam süreye böldüğünüzde en fazla ne kadar hızlanabileceğiniz çıkar:

$$\text{hızlanma} \leq \frac{1}{S + \dfrac{1-S}{N}}$$

7.2 Derste Verilen Örnek

Verilen: Programın %25'i seri, %75'i paralel. İşlemci sayısı 1'den 2'ye çıkarılıyor. Yani $S = 0.25$, $N = 2$.

- Paralel terim: $\dfrac{1-S}{N} = \dfrac{0.75}{2} = 0.375$ (ara adım eklendi).
- Payda: $S + 0.375 = 0.25 + 0.375 = 0.625$ (ara adım eklendi).
- Hızlanma: $\dfrac{1}{0.625} = 1.6$.

Sonuç: İşlemci sayısını iki katına çıkarmak programı 2 kat değil, 1.6 kat hızlandırır. Hesap Python ile doğrulandı.

7.3 Çekirdek Sayısı Çok Büyürse

Hoca, N'in "fantastik" sayılara çıkarıldığı durumu da gösterdi. Slaytta sonsuz olarak yazılmıştı, ama sonsuz olması gerekmez; $N = 1000$ bile olsa paralel terim küçük bir sayıya gider ve etkisi iyice azalır. Bu durumda formül yaklaşık olarak $1/S$ şeklinde ifade edilebilir.

> **Ders sonrası not:** Derste "küçük bir sayıya gider" denen şey hızlanmanın kendisi değil, paralel terim $\frac{1-S}{N}$'dir. Hızlanma ise $\frac{1}{S}$'ye yaklaşır. Derste verilen $S = 0.25$ için bu sınır $\frac{1}{0.25} = 4$'tür (ara adım eklendi). Yani bu program kaç çekirdek eklenirse eklensin 4 kattan fazla hızlanamaz.

8. Kullanıcı Seviyesi ve Çekirdek Seviyesi İş Parçacıkları

Bir sistem iş parçacığı destekleyecekse, süreçlerin içinde birden fazla akış olmasını sağlayan bir mekanizma sunması gerekir. Bilindiği gibi iki çalışma kipimiz var: kullanıcı kipi (user mode) ve çekirdek kipi (kernel mode). Bu yüzden destek iki yerde sağlanabilir:

- **Kullanıcı seviyesi (user-level):** Destek genelde uygulamaya doğrudan eklenen bir kütüphane ile sağlanır. Bütün sorumluluk ve yönetim kütüphane tarafındadır. Hoca örnek olarak POSIX Pthreads'i, Windows iş parçacıklarını ve Java iş parçacıklarını saydı.
- **Çekirdek seviyesi (kernel-level):** İş parçacıklarının oluşturulması ve yönetimiyle ilgili tüm sorumluluk işletim sistemi çekirdeğindedir.

  > **Ders sonrası not:** Hoca bu noktada Windows iş parçacıklarının da kullanıcı seviyesinde sağlandığını söyledi. Bir sonraki konuda ise Windows iş parçacığı kütüphanesinin doğrudan çekirdekte gerçeklendiğini söyledi. Doğrusu ikincisidir. Kullanıcı ya da çekirdek seviyesinde gerçeklenebilen kütüphane Pthreads'tir.

İş parçacığı yapısı ilk ortaya çıktığında genellikle kullanıcı seviyesinde sunulmuştu. Günümüzde ise bütün işletim sistemleri (Windows, Solaris, macOS, Linux) çekirdek seviyesinde iş parçacığını standart olarak destekler.

Her iki durumda da kullanıcı seviyesi ile çekirdek seviyesi arasında bir eşleşme yapısı gerekir. Destek bir kütüphane ile kullanıcı seviyesinde sağlanıyorsa işletim sisteminin altta buna uygun bir mekanizması olmalıdır. İşletim sistemi doğrudan destekliyorsa da bunun kullanıcı kipinde bir parçası olmalıdır. Bu eşleşme için kullanılan yapılar bir sonraki konudaki modellerdir.

9. Çoklu İş Parçacığı Modelleri

Üç temel model vardır: çoktan teke (many-to-one), teke tek (one-to-one) ve çoktan çoka (many-to-many). Hoca bir dördüncü modelden de bahsetti: Yer yer iki seviyeli (two-level), yer yer hibrit diye geçer ve bu üç modelden ikisinin birlikte kullanılmasıdır.

9.1 Çoktan Teke

Kullanıcı seviyesindeki birden fazla iş parçacığı çekirdek seviyesinde tek bir iş parçacığına denk düşer. Slayttaki örnekte dört kullanıcı iş parçacığı vardı.

En büyük sorun bloklanmadır. Bu dört iş parçacığından herhangi biri herhangi bir anda kendisini bloklayacak bir noktaya gelebilir: bir kilitte (lock) bekleyebilir ya da beklemeye neden olacak bir işlem başlatabilir. Bu modelde çekirdek iş parçacığı o kullanıcı iş parçacığının işini yürüttüğü için o da bloklanır. Bu, diğer bütün iş parçacıklarının da beklemek zorunda kalması demektir. Çünkü onları çalıştırıp ilerletebilecek başka bir çekirdek iş parçacığı yoktur.

9.2 Teke Tek

Her kullanıcı iş parçacığı çekirdek seviyesinde bir iş parçacığıyla eşleştirilir. Bu, çoktan teke modelindeki kısıt açısından çok güzeldir: Biri bloklanınca diğerleri etkilenmez.

Tek sorunu maliyettir. Çoktan tekede 4 kullanıcı iş parçacığı için 1 çekirdek iş parçacığı oluşturuyorduk. Burada 4'ü için 4 tane oluşturuyoruz. Bu ek maliyet yüzünden işletim sistemleri genellikle aynı anda var olabilecek iş parçacığı sayısını sınırlar.

9.3 Çoktan Çoka

Alternatif yaklaşım eşlemeyi çoktan çoka yapmaktır. Bu model temelde esneklik sağlar. Slayttaki örnekte kullanıcı seviyesinde 4, çekirdek seviyesinde 3 iş parçacığı vardı. Hoca çekirdek tarafındaki sayının 2 ya da 3 olabileceğini, 5 olmasının biraz tuhaf olacağını söyledi.

Bunu bir çekirdek iş parçacığı havuzu gibi düşünebilirsiniz. Kullanıcı iş parçacıklarından biri bloklanan bir duruma girince havuzdaki çekirdek iş parçacıklarından yalnız biri bloklanır. Kalanlar diğer iş parçacıklarının çalışmasını sürdürür. Böylece çoktan teke modelindeki temel sorun ortadan kalkar. Eşleme birebir olmadığı için kaynak kullanımı da teke teke göre daha etkindir.

9.4 İki Seviyeli (Hibrit) Model

Çoktan çoka ve teke tek modelleri bir araya getirilerek iki seviyeli ya da hibrit bir model oluşturulabilir. Bu modelin amacı derste öğrencilere soru olarak soruldu (bkz. Örnek Sorular). Cevap: Daha önemli ya da yüksek öncelikli iş parçacıklarını doğrudan işletim sistemindeki bir çekirdek iş parçacığıyla ilişkilendirebilmektir. Diğer iş parçacıkları ise çoktan çoka biçiminde eşlenmeye devam eder.

10. İş Parçacığı Kütüphaneleri

İş parçacıklarını oluşturmak, sonlandırmak ve yönetmek için bir programlama arayüzüne (API) ihtiyaç vardır. Bu API bir iş parçacığı kütüphanesi (thread library) ile gerçeklenir. Kütüphane iki şekilde gerçeklenebilir:

- **Kullanıcı uzayında:** Kütüphanenin hem kodu hem de kullandığı veri yapıları tamamen kullanıcı uzayındadır. İşletim sistemi çekirdeğinin kütüphaneyle doğrudan bir etkileşimi yoktur.
- **Çekirdek uzayında:** Kütüphane doğrudan çekirdek tarafından desteklenir. Kütüphanenin kodu ve veri yapıları çekirdektedir. Kütüphanedeki bir fonksiyonu kullandığınızda, kullanıcı uzayında ona denk düşen yerel fonksiyonu çağırırsınız. Bu fonksiyon, sistem çağrısı arayüzü üzerinden çekirdekteki ilgili fonksiyonu etkin hale getirir. Hoca bunun, C'de bir çalışma zamanı kütüphanesinden (runtime) bir fonksiyon çağırmaya çok benzediğini söyledi.

Kullanılan üç temel kütüphane şunlardır:

- **POSIX Pthreads:** POSIX standardının bir uzantısıdır. Üzerinde çalıştığınız sisteme bağlı olarak kullanıcı uzayında da çekirdek uzayında da gerçeklenmiş olabilir.
- **Windows iş parçacığı kütüphanesi:** Doğrudan işletim sistemi çekirdeğinde gerçeklenmiştir.
- **Java iş parçacığı kütüphanesi:** Java programları Java sanal makinesi (JVM) üzerinde çalışır. JVM, üzerinde çalıştığı işletim sisteminin sunduğu kütüphaneyle etkileşerek bu desteği sağlar. Windows bir makinede Windows kütüphanesini, UNIX ya da türevi bir sistemde Pthreads kütüphanesini kullanır.

Derste ve dersin uygulamalarında Pthreads üzerinden gidilecek. Hoca Pthreads'in bir standart, bir belirtim (specification) olduğunu hatırlattı. Genelde UNIX türevi işletim sistemlerinde POSIX standardının parçası olarak gerçeklenir. Belirtim neyin nasıl olması gerektiğini tarif eder, ama bu tarifte mutlaka açık uçlu yerler vardır. Bu yüzden gerçeklemeler arasında yer yer farklılık görülebilir. Fakat fonksiyonların davranışlarının her zaman aynı olması beklenir.

11. Pthreads Örneği: 1'den N'ye Toplam

11.1 Programın Çalışması

Hoca slayttaki basit Pthreads örneğini anlattı. Programın bazı kısımlarının çok önemli olmadığını, ama şunların bilinmesi gerektiğini söyledi:

- Her sürecin bir kimliği olduğu gibi her iş parçacığının da bir kimliği (`pthread_t`) vardır.

- İş parçacığı oluşturulurken onun özelliklerini belirttiğiniz bir yapıya (`pthread_attr_t`) ihtiyaç vardır. Bunu kullanmak her zaman zorunlu değildir. Ama özel bir durum belirtecekseniz kullanmanız gerekir.

Program komut satırından iki argüman bekler: programın kendi adı ve bir tam sayı. Argüman sayısı ikiden farklıysa çalışmaz. Verilen tam sayı, oluşturulan iş parçacığına argüman olarak gönderilir. Program bu değerin sıfırdan küçük olmamasını ister. Hoca bu kısıtın neden konduğunu bilmediğini, tamamen uygulamaya bağlı bir kısım olduğunu söyledi.

Akış şöyledir:

1. `pthread_attr_init` ile oluşturulacak iş parçacığının özellik yapısı varsayılan değerlerle doldurulur.
2. `pthread_create` ile yeni bir iş parçacığı oluşturulur. Fonksiyona sırasıyla iş parçacığı kimliğinin yazılacağı yer, özellik yapısı, iş parçacığının çalıştıracağı fonksiyon ve o fonksiyonun parametresi verilir.
3. Yeni iş parçacığı `runner` fonksiyonunu çalıştırır: Global `sum` değişkenini önce sıfırlar, sonra 1'den verilen değere kadar bütün sayıları toplar.
4. Ana iş parçacığı `pthread_join` ile yeni iş parçacığının bitmesini bekler ve `sum` değerini ekrana yazar.

Derleyiciyle sınanan kod (slayttaki kitap örneğinin yapısı korunmuştur):

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum; /* iş parçacıkları arasında ortak veri */
void *runner(void *param); /* iş parçacığının çalıştıracağı fonksiyon */

int main(int argc, char *argv[])
{
    pthread_t tid;       /* iş parçacığı kimliği */
    pthread_attr_t attr; /* iş parçacığı özellikleri */

    if (argc != 2) {
        fprintf(stderr, "kullanim: a.out <tamsayi>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d >= 0 olmali\n", atoi(argv[1]));
        return -1;
    }

    pthread_attr_init(&attr);                       /* varsayılan özellikler */
    pthread_create(&tid, &attr, runner, argv[1]);  /* iş parçacığını oluştur */
    pthread_join(tid, NULL);                        /* bitmesini bekle */

    printf("sum = %d\n", sum);
    return 0;
}

void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;
    for (i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
```

`gcc -Wall -pthread` ile uyarısız derlendi. `./a.out 10` için `sum = 55`, `./a.out 100` için `sum = 5050` yazdı. Negatif argümanla hata iletisi verip çıktı.

> **Ders sonrası not:** Hoca önce değerin "sıfırdan küçük olmaması", sonra "sıfırdan büyük pozitif bir değer olması" gerektiğini söyledi. Koddaki denetim yalnız negatif değerleri reddeder. 0 kabul edilir ve sonuç `sum = 0` olur.

11.2 İş Parçacığı Fonksiyonunun Tipi

İş parçacığı oluşturulurken çalıştıracağı fonksiyon ve o fonksiyonun parametresi verilir. Bu fonksiyonun dönüş tipi ve parametresi `void *` olmalıdır. Neden `void *`? C'de fonksiyonu `int` ya da başka bir tiple tanımlarsanız oraya başka bir şey gönderme ya da tip dönüşümü yapma şansınız olmaz. `void *` ise istediğiniz türe dönüştürebileceğiniz bir işaretçidir.

Tek bir parametreyle sınırlısınız. Birden fazla parametre göndermek için bir yapı (`struct`) oluşturup içini doldurur ve yapının adresini gönderirsiniz.

Hoca bu noktada `argv[1]`'in veri tipini sordu. Cevap `char *`'dır (bkz. Örnek Sorular). `atoi` ile yapılan da bu karakter dizisinden bir tam sayı elde etmektir. Kullanıcının 15, 20, 25 gibi bir değer vermesi beklenir.

> **Hocanın vurgusu:** Standart C'de `string` diye bir veri tipi yoktur. Bu kod C++ değil, standart C kodudur. Olmayan veri tiplerini uydurmayın.

11.3 İş Parçacığını Bitirmek: `pthread_exit` ve `exit`

`runner` fonksiyonu `pthread_exit` ile biter. Bunun yerine `return` da kullanılabilir, fark etmez.

> **Hocanın vurgusu:** İş parçacığı fonksiyonunun içinde `exit()` çağırırsanız bütün süreci yok edersiniz. Daha önce de söylendiği gibi ortalık karışır. Dikkat edin.

11.4 Global Değişken ve Ortak Alan

Örnekte `sum` global bir değişkendir. Böylece ortak alan paylaşımının nasıl olduğu, global değişkenlere bütün iş parçacıklarının erişebildiği görülür. Bu iş parçacığından bir tane daha oluşturup aynı şeyleri yapmaya kalkarsanız yine ortalık karışabilir. Bu yüzden ortak alana erişim kontrollü yapılmalıdır.

11.5 Birleştirilebilir ve Ayrık İş Parçacıkları

İş parçacıklarının çalışma biçimi birleştirilebilir (joinable) ya da ayrık (detached) olabilir. Bir süreç bir iş parçacığı oluşturduğunda onun ya da bir iş parçacığı grubunun bitmesini bekleyebilir. Ya da onlarla aynı anda çalışmaya devam edip bağımsız çalışmalarını sağlayabilir. Bu, genelde iş parçacığı oluşturulurken önceden belirtilir. Birleştirileceği belirtildiyse `pthread_join` ile beklenmesi gerekir. Hoca ayrıntıların uygulamada görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Pthreads'te iş parçacıkları varsayılan olarak birleştirilebilir oluşturulur. Ayrık oluşturmak için özellik yapısında `pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED)` çağrılabilir ya da çalışan bir iş parçacığı sonradan `pthread_detach` ile ayrık yapılabilir. Ayrık bir iş parçacığı bittiğinde kaynakları kendiliğinden geri verilir ve `pthread_join` ile beklenemez.

12. Windows ve Java İş Parçacıkları

12.1 Windows

Aynı uygulamanın Windows'taki gerçeklemesi de çok benzerdir. Yine bir fonksiyon ve yine `Sum` adında ortak bir veri alanı vardır. Değişen yalnızca API'nin sunduğu fonksiyonların adlarıdır. Dolayısıyla ortada büyük bir fark yoktur.

12.2 Java

Java'da iş parçacığı oluşturmanın iki yolu vardır:

- `Thread` sınıfından yeni bir sınıf türetmek ve bu sınıfta `run` adlı metodun içeriğini doldurmak.
- Daha yaygın olan yol: `Runnable` arayüzünü (interface) iş parçacığı oluşturacak sınıfa uygulamak ve bu arayüzün sunduğu `run` metodunun içeriğini doldurmak.

Slayttaki en basit örnekte `Runnable` uygulanmış ve `run` metodunun içi doldurulmuştu.

13. Örtük İş Parçacığı Kullanımı

Buraya kadar her şey programcının kontrolündeydi: İş parçacıklarının oluşturulması, senkronizasyonu ve benzeri işler programcı tarafından yönetiliyordu. Pthreads örneğinde gördüğünüz gibi iş parçacığının nerede, nasıl oluşturulacağını, nasıl haberleşeceğini, ne üzerinde çalışacağını hep biz söylüyoruz.

Alternatif olarak iş parçacıklarının yönetimini bizim yerimize yapan üçüncü taraf yapılardan faydalanabiliriz. Buna örtük iş parçacığı kullanımı (implicit threading) denir. Bu yaklaşımda iş parçacıklarının oluşturulması ve yönetimi ya çalışma zamanı kütüphanesine (runtime library) ya da derleyiciye bırakılır.

- **İş parçacığı havuzu (thread pool):** Hoca bunun biraz farklı bir yapı olduğunu söyledi. Önceden belli sayıda iş parçacığı oluşturulur. Hangi iş parçacığının hangi görevi yapacağı dinamik olarak söylenir ve arka planda yönetilir. Belli açılardan hâlâ ilk konuştuğumuz programcı kontrollü yönteme benzer.
- **OpenMP:** Farklı derleyiciler için desteği ve API'si vardır. Kod içinde hangi parçanın paralelleştirileceği `#pragma` yönergesiyle söylenir. Programcının yaptığı yalnız budur, gerisini derleyici gerçekleştirir.
- **Grand Central Dispatch:** macOS tarafında kullanılan bir yapıdır. Burada da kodun belli bir kısmı işaretlenir. İşaretlenen kısmın çalışma zamanı kütüphanesi tarafından doğrudan çok iş parçacıklı çalıştırılması beklenir.

Hoca bu kısımların ayrıntısının çok önemli olmadığını söyledi.

> **Ek bilgi (derste anlatılmadı):** OpenMP'de en sık görülen yönerge `#pragma omp parallel for` biçimindedir; ardından gelen `for` döngüsünün yinelemeleri iş parçacıklarına paylaştırılır. GCC'de OpenMP desteği `-fopenmp` seçeneğiyle açılır.

14. İş Parçacığı Kullanımındaki Sorunlar

Hoca, iş parçacığı kullanıldığında özellikle sistem çağrıları çevresinde bazı sorunlar çıktığını hatırlattı ve konuşulacakları sıraladı:

- `fork` ve `exec` çağrılarının anlamı.
- Sinyallerin (signals) nasıl teslim edileceği ve işleneceği. Hoca sinyallerin daha önce yalnız kısaca, ikinci bölüm slaytlarında konuşulduğunu söyledi.
- İş parçacığı iptali (thread cancellation): Süreçlerdeki olağan dışı sonlanmaya benzer şekilde bir iş parçacığı iptal edilebilmelidir. Bunun asenkron ya da ertelenmiş yapılması tasarım kararıdır.
- İş parçacığına özel depolama (thread-local storage) alanının yönetimi.
- Çoktan çoka yapıda kullanıcı ve çekirdek iş parçacıklarının eşleşmesini, kimin kimi ne zaman çalıştıracağını düzenleyen yöntem.

14.1 `fork()` ve `exec()`

`fork` sistem çağrısı normalde bir sürecin birebir kopyasını oluşturur. Şimdi içinde birden fazla iş parçacığı olan bir süreç düşünün. Bu iş parçacıklarından biri `fork`'u çağırdı. Sonuç ne olabilir?

- Yeni süreç, eski süreçteki bütün iş parçacıklarıyla birlikte oluşturulabilir. Örneğin dört iş parçacığı varsa yeni süreç de dört iş parçacığı içerir.
- Ya da yeni süreç yalnız tek bir iş parçacığıyla oluşturulabilir.

Hangisinin daha faydalı olduğunu programcının amacı belirler. Programcının amacı yeni bir süreç oluşturup ona `exec` ile başka bir program yükleyip çalıştırmaksa, `fork`'un diğer iş parça-

cıklarını kopyalamadan tek iş parçacıklı bir kopya oluşturması daha mantıklıdır. Öte yandan aynı işleri yapacak, aynı iş parçacıklarına sahip bir süreç de istenebilir. İşletim sisteminin hangisinin istendiğini doğrudan bilme şansı yoktur. Bu yüzden birçok UNIX sisteminde `fork` sistem çağrısının iki farklı sürümü vardır. Biri sürecin kopyasını yalnızca ana iş parçacığıyla alır, diğeri bütün iş parçacıklarıyla birlikte alır. Programcı hangisini istiyorsa onu çağırır.

`exec` için benzer bir sorun olabilirdi, ama burada bir kısıt durumu kolaylaştırır. `exec` ile sürecin içine doğrudan yeni bir program yüklenir. İçinde bir ya da birden fazla iş parçacığı olması fark etmez, yapılacak şey bütün içeriği silip yeniden doldurmaktır. Bu yüzden `exec` için böyle bir ayrıma gitmenin pek anlamı yoktur.

> **Ders sonrası not:** Hoca, `fork`'un birinci sürümünün "yalnızca ana iş parçacığıyla" kopya aldığını söyledi. POSIX'te tek iş parçacıklı kopya, ana iş parçacığını değil `fork`'u çağıran iş parçacığını içerir. Linux'ta ve POSIX'te `fork()` yalnızca çağıran iş parçacığını kopyalar. Bütün iş parçacıklarını kopyalayan sürüm büyük olasılıkla yalnız bazı eski UNIX sistemlerinde (Solaris'teki `forkall()` gibi) bulunur.

14.2 Sinyal İşleme

Temel sorun şudur: Bir sinyal oluşturulur ve hedef sürece gider. Ama hedef süreçte birden fazla akış, yani birden fazla iş parçacığı vardır. Sinyal bu iş parçacıklarından hangisine gidecek ve hangisine gittiğine bağlı olarak nasıl bir işlem yapılacak? İşletim sistemi olarak olası seçenekler şunlardır:

- Sinyali süreçteki bütün iş parçacıklarına dağıtmak.
- Yalnızca bir iş parçacığına göndermek. Bu, o anda seçilen herhangi bir iş parçacığı ya da daha önceden bu iş için atanmış bir iş parçacığı olabilir.
- Belli bir grup iş parçacığına göndermek.

Tek bir iş parçacığına gönderildiğinde pek sorun yoktur. Sinyale karşı tanımlanmış bir sinyal işleyici (signal handler) vardır, oradaki işlemler yapılır ve her şey akışında gider. Hoca bunu en temiz senaryo olarak niteledi. Fakat sinyal bir gruba ya da bütün iş parçacıklarına gönderilirse sinyali alan her iş parçacığı işleyiciyi çalıştırmaya kalkar. Böylece aynı işleyicinin birden fazla kez çalışması sorunu çıkar.

Kısacası iki soru vardır: Sinyal kime dağıtılacak ve sinyali alan iş parçacığı ne yapacak?

14.3 İş Parçacığı İptali

Oluşturduğunuz bir iş parçacığını herhangi bir anda iptal etmek isteyebilirsiniz. Hocanın canlı senaryosu bir arama programıydı: Bilgisayar içinde ya da bir sözlükte arama yapıyorsunuz ve bunu etkin olsun diye çok iş parçacıklı yapıyorsunuz. İş parçacıklarından biri istenen sonucu buldu. Diğerlerinin çalışmaya devam etmesine gerek yok, onları kapatmak istersiniz. İptal edilecek iş parçacığına hedef iş parçacığı (target thread) denir. Bir iş parçacığının kendi isteği dışında sonlandırılmasına iş parçacığı iptali denir. İki şekilde yapılabilir:

Asenkron iptal (asynchronous cancellation). İstediğiniz bir anda hedef iş parçacığına iptal isteği gönderirsiniz. Pthreads üzerinden konuşursak bu `pthread_cancel` gibi bir çağrıyla yapılır. Hedef iş parçacığı o anda ortadan kaldırılır. Temel sorun şudur: İş parçacığı doğrudan kaldırıldığı için sahip olduğu kaynaklar işletim sistemine düzgünce iade edilmemiş olur.

İş parçacığına özel depolama ile ilişkisi. Hoca burada biraz öne atlayıp TLS kavramını açtı (bkz. 14.4). İş parçacığının yığınındaki yerel değişkenler zaten geçicidir. İş parçacığı iptal edilince bunlar ortadan kalkar, "iptal ettim ama orada veri kaldı" diye bir durum yoktur. Hoca, sorunun TLS alanında olduğunu söyledi: Özellikle asenkron iptalde TLS alanı doğrudan ortadan kalkmaz ve bu dikkat edilmesi gereken bir durumdur.

> **Ders sonrası not:** Hoca bu kısmı anlatırken önce TLS alanının iptalde "ortadan doğrudan kalkmadığını", hemen sonra "TLS kalktı" dedi; anlatım burada karıştı. Asenkron iptaldeki asıl sorun, iş parçacığının elindeki kaynakların (tuttuğu kilitler, ayırdığı bellek, açtığı dosyalar gibi) düzgün geri verilmemesidir. Hocanın ilk söylediği de buydu.

Ertelenmiş iptal (deferred cancellation). Asenkron iptal yerine kullanılan mekanizmadır. İptal isteği gönderilir, ama hedef iş parçacığı o anda kaldırılmaz. Hedef iş parçacığı periyodik olarak, belli noktalarda kendisiyle ilgili bir iptal isteği olup olmadığına bakar. İstek varsa bu ara kontrollerden birinde bunu fark eder ve kendini iptal eder. Bu çok daha temiz bir mekanizmadır. Ama iptal isteğinin gönderilmesi ile gerçekten yapılması arasında bir gecikme vardır. "Ertelenmiş" adı buradan gelir.

> **Ek bilgi (derste anlatılmadı):** Pthreads'te varsayılan iptal türü ertelenmiş iptaldir. İptal isteği yalnız iptal noktalarında (cancellation point) işlenir. `pthread_testcancel()` çağrısı açık bir iptal noktası oluşturur, `read` ve `sleep` gibi bloklanabilen birçok çağrı da iptal noktasıdır. Asenkron iptal `pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, ...)` ile seçilir.

14.4 İş Parçacığına Özel Depolama

İş parçacıkları içinde bulundukları sürecin ortak alanlarını kullanır. Bunun dışında kendi yığınlarında yerel değişkenleri vardır, ama bunlar geçicidir. Uygulamanın yapısına bağlı olarak her iş parçacığının yalnız kendisinin erişebileceği bir veri alanına sahip olmasını isteyebiliriz. Bu iş parçacığına özel veri alanına iş parçacığına özel depolama (thread-local storage, TLS) denir.

> **Hocanın vurgusu:** TLS, yerel değişkenlerle karıştırılmamalıdır. Bambaşka bir şeyden bahsediyoruz.

TLS'yi işletim sistemi, iş parçacığı desteğini hangi kütüphane üzerinden sağlıyorsa onun üzerinden gerçekler. POSIX'te, Windows'ta ve Java sanal makinesinde TLS alanı oluşturmaya izin verilir.

14.5 Hafif Süreç ve Zamanlayıcı Aktivasyonları

Çoktan çoka yapılarda ve onu içeren iki seviyeli yapılarda birden fazla kullanıcı iş parçacığı ile işletim sisteminin birden fazla iş parçacığı arasında geçiş yapılır. Bu geçişi kontrol edecek, mekanizmayı ayarlayacak bir yapıya ihtiyaç vardır. Bu yapıya genellikle hafif süreç (lightweight process, LWP) denir.

Kullanıcı seviyesindeki iş parçacığı kütüphanesi LWP'yi sanal bir işlemci olarak görür. Çalıştırılmasına karar verdiği kullanıcı iş parçacığını bu sanal işlemciyle eşleştirir. Fakat LWP'nin de çalışabilmesi için çekirdek tarafında bir iş parçacığına denk düşürülmesi gerekir.

Sorun şudur: LWP'nin denk düştüğü çekirdek iş parçacığı bloklanırsa ne olur? Aradaki geçişler yapılamaz, yani kullanıcı seviyesinde gerçeklenmiş kütüphane iş parçacıkları arasında geçiş yapamaz.

Hoca çözümü "kaba olacak ama bir tür hülle yöntemi" diye anlattı: Bloklanma olursa bir çekirdek iş parçacığı daha oluşturulur. LWP bu yeni çekirdek iş parçacığıyla eşleştirilir ve geçişi yapması sağlanır. Sonrasında bu iş parçacığı devreden çıkarılır. Böylece esnek bir yapı oluşur ve bloklanma en aza indirilir.

> **Ek bilgi (derste anlatılmadı):** Kitapta bu düzen zamanlayıcı aktivasyonu (scheduler activation) adıyla geçer. Çekirdek, bir iş parçacığının bloklanacağını uygulamaya yukarı çağrı (upcall) ile bildirir ve ona yeni bir LWP ayırır. Kütüphane, yukarı çağrı işleyicisini bu yeni LWP üzerinde çalıştırır, bloklanan iş parçacığının durumunu kaydeder ve başka bir kullanıcı iş parçacığını çalıştırır.

15. Örnek Sorular

Bu derste ekrandan alınmış kare yoktur. Aşağıdaki sorular derste hocanın sözlü sorduğu sorulardır.

Soru 1 (derste sözlü, 00:33:14): Tek işlemcili bir sisteme bir işlemci daha eklendiğinde, program buna uygun yazılmış olsa bile başarım neden her zaman iki katına çıkmaz? Programın %25'i seri, %75'i paralelse 2 işlemciyle hızlanma ne olur?

Çözüm (derste verildi): Amdahl Yasası'na göre seri kısım bölünemez. $S = 0.25$, $N = 2$ için hızlanma $\frac{1}{0.25 + \frac{0.75}{2}} = \frac{1}{0.625} = 1.6$ olur. Program 2 kat değil 1.6 kat hızlanır. Çekirdek sayısı çok büyüdükçe hızlanma $1/S$'ye yaklaşır.

Soru 2 (derste sözlü, 00:43:56): Çoktan çoka ve teke tek modellerini birleştiren iki seviyeli (hibrit) modelde amaç nedir? Şekilde sol ve sağ taraftaki kullanıcı iş parçacıkları arasında nasıl bir fark vardır?

Çözüm (derste verildi): Fark önem derecesindedir. İki seviyeli model, daha önemli ya da yüksek öncelikli iş parçacıklarını doğrudan bir çekirdek iş parçacığıyla ilişkilendirme şansı verir. Diğer iş parçacıkları çoktan çoka biçiminde eşlenir.

Soru 3 (derste sözlü, 00:53:21): Pthreads örneğinde `pthread_create`'e parametre olarak `argv[1]` gönderiliyor. `argv[1]`'in veri tipi nedir?

Çözüm (derste verildi): `char *`, yani bir karakter işaretçisi. Derste "string pointer" gibi bir cevap verildi; hoca, standart C'de `string` diye bir tip olmadığını ve kodun C++ değil standart C olduğunu söyleyerek düzeltti. Kodda `atoi` ile bu karakter dizisinden tam sayı elde edilir.

Soru 4 (derste sözlü, 01:04:54): Çok iş parçacıklı bir süreçte iş parçacıklarından biri `fork()` çağırırsa sonuç ne olabilir? Hangisi daha doğrudur?

Çözüm (derste verildi): Yeni süreç ya bütün iş parçacıklarıyla ya da yalnız tek iş parçacığıyla oluşturulabilir. Hangisinin doğru olduğunu programcının amacı belirler. `fork`'tan sonra `exec` ile başka bir program yüklenecekse tek iş parçacıklı kopya daha mantıklıdır. İşletim sistemi amacı bilemediği için birçok UNIX sisteminde `fork`'un iki sürümü vardır.

Dersin Toparlaması

Bu derste süreç oluşturmanın maliyetinden yola çıkılarak iş parçacığı kavramı kuruldu. Bir akış için yalnız program sayacı, yazmaçlar ve yığın gerektiği için iş parçacıkları yalnız bunları ayrı tutar, kodu, veriyi, yığın belleği ve açık dosyaları süreçle paylaşır. Bu paylaşım duyarlılık, ekonomi, ölçeklenebilirlik ve doğrudan kaynak paylaşımı getirir, ama ortak alana erişimin düzenlenmesi programcının sorumluluğuna kalır. Çok çekirdekli sistemlerde veri ve görev paralelliğiyle iş bölünebilir. Amdahl Yasası ise seri kısmın hızlanmaya bir üst sınır koyduğunu gösterir. Kullanıcı ve çekirdek iş parçacıklarının eşlenmesi için çoktan teke, teke tek, çoktan çoka ve iki seviyeli modeller incelendi. Pthreads ile 1'den N'ye toplam örneği üzerinden `pthread_create`, `pthread_join`, `void *` parametre ve `pthread_exit`/`exit` farkı görüldü.

Son olarak örtük iş parçacığı yaklaşımları (iş parçacığı havuzu, OpenMP, Grand Central Dispatch) ve iş parçacığı kullanımının getirdiği sorunlar konuşuldu: `fork`/`exec` anlamı, sinyallerin teslimi, asenkron ve ertelenmiş iptal, TLS ve hafif süreçler. Bölümün sonundaki işletim sistemi örneklerine derste girilmedi. Hoca, bu konuların dersin uygulamasında Linux üzerinde Pthreads kütüphanesiyle gerçekten uygulanarak görüleceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.